# KUBERENETS -05

## Task 1: Namespace and ResourceQuota

- Created `dev-environment` namespace.
- Applied ResourceQuota to control namespace resources.
- Restricted maximum Pods to 3.
- Restricted maximum Services to 2.
- Verified quota using `kubectl describe`.

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"
~
~
~
~
```

```
root@k8s-master:~# kubectl create namespace dev-environment
namespace/dev-environment created
root@k8s-master:~# kubectl get namespaces
NAME              STATUS   AGE
default           Active   5d
dev-environment   Active   14s
kube-flannel      Active   4d23h
kube-node-lease   Active   5d
kube-public       Active   5d
kube-system       Active   5d
root@k8s-master:~# vi resource-quota.yaml
root@k8s-master:~# kubectl apply -f resource-quota.yaml
resourcequota/dev-quota created
root@k8s-master:~# kubectl get pods
No resources found in default namespace.
root@k8s-master:~# kubectl get resourcequota -n dev-environment
NAME        REQUEST                       LIMIT   AGE
dev-quota   pods: 0/3, services: 0/2              28s
root@k8s-master:~# kubectl describe resourcequota dev-quota -n dev-environment
Name:       dev-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
pods        0     3
services    0     2
root@k8s-master:~#
```

# Task 2: Pod with CPU and Memory Requests and Limits

## OBJECTIVE: CREATE A POD IN PROD-ENVIRONMENT WITH:

| Resource | Request | Limit |
|---|---|---|
| CPU | 200m (0.2) | 500m (0.5) |
| Memory | 200Mi | 500Mi |

- Created `prod-environment` namespace.
- Configured CPU and memory requests.
- Configured CPU and memory limits.
- Verified resource allocation using `kubectl describe pod`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
  namespace: prod-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "200m"
        memory: "200Mi"
      limits:
        cpu: "500m"
        memory: "500Mi"
```

```
root@k8s-master:~# kubectl create namespace prod-environment
namespace/prod-environment created
root@k8s-master:~# vi prod-pod.yaml
root@k8s-master:~# kubectl apply -f prod-pod.yaml
pod/resource-pod created
root@k8s-master:~# kubectl get pods -n prod-environment
NAME           READY   STATUS    RESTARTS   AGE
resource-pod   1/1     Running   0          10s
root@k8s-master:~# kubectl describe pod resource-pod -n prod-environment
Name:             resource-pod
Namespace:        prod-environment
Priority:         0
Service Account:  default
Node:             k8s-worker2/172.31.28.79
Start Time:       Wed, 11 Nov 2026 10:22:18 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.1.79
IPs:
  IP:  10.244.1.79
Containers:
  nginx:
```

```
Containers:
  nginx:
    Container ID:   containerd://7dd517a5e81e46176b204bea178ac953f8585efbd11eaad6ab1d89c6686ba52
b
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56
caf67512486aff76d82
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 11 Nov 2026 10:22:19 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:     500m
      memory:  500Mi
    Requests:
      cpu:        200m
      memory:     200Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-fhxnq (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
```

# Task 3: LimitRange with Default CPU and Memory

## OBJECTIVE: IN STAGING-ENVIRONMENT, CONFIGURE DEFAULT RESOURCE LIMITS:

- Default CPU: 300m
- Default Memory: 600Mi
- Minimum and maximum CPU values.

For this practical, we are using:

- Minimum CPU: 100m
- Maximum CPU: 500m

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limits
  namespace: staging-environment
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: "600Mi"
    min:
      cpu: "100m"
    max:
      cpu: "500m"
~
~
~
```

```
root@k8s-master:~# kubectl create namespace staging-environment
namespace/staging-environment created
root@k8s-master:~# vi staging-limitrange.yaml
root@k8s-master:~# kubectl apply -f staging-limitrange.yaml
limitrange/staging-limits created
root@k8s-master:~# kubectl describe limitrange staging-limits -n staging-environment
Name:       staging-limits
Namespace:  staging-environment
Type        Resource  Min   Max   Default Request  Default Limit  Max Limit/Request Ratio
----        --------  ---   ---   ---------------  -------------  -----------------------
Container   cpu       100m  500m  300m             300m           -
Container   memory    -     -     600Mi            600Mi          -
```

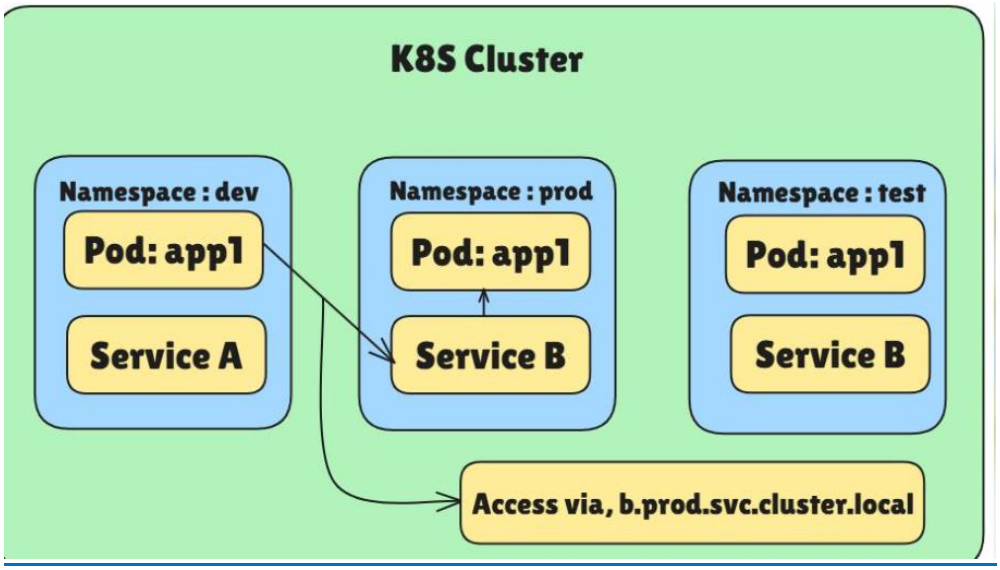
```
 memory limit for container staging-test
Status:           Running
IP:               10.244.2.87
IPs:
  IP:  10.244.2.87
Containers:
  staging-test:
    Container ID:   containerd://845654a9b0971583c8aea1d90339fe35f5350a8eab323f1890dfcef6bc8ce14
2
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56
caf67512486aff76d82
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 11 Nov 2026 10:33:01 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:     300m
      memory:  600Mi
    Requests:
      cpu:        300m
      memory:     600Mi
    Environment:  <none>
    Mounts:
```

- Created `staging-environment` namespace.
- Applied LimitRange policy.
- Configured default CPU limit of 300m.
- Configured default memory limit of 600Mi.
- Set minimum CPU to 100m and maximum CPU to 500m.
- Verified default limits using a test Pod.

# Task 4: Pod Communication Using Service DNS

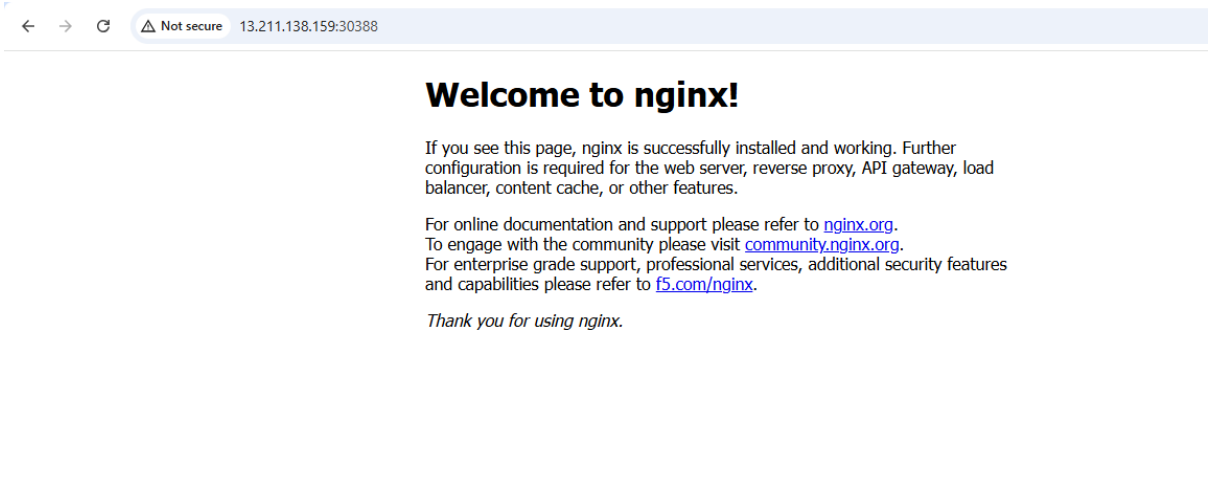
Objective:

- Create a Pod and NodePort Service in `default` namespace.
- Create another Pod in `test` namespace.
- Communicate between Pods using Service DNS.
- `kubectl exec` lets you execute commands inside a container.
- `curl` tests HTTP connectivity.
- Kubernetes Service DNS enables communication across namespaces.




```
root@k8s-master:~# kubectl create namespace test
namespace/test created
root@k8s-master:~# kubectl run nginx-pod --image=nginx --port=80
pod/nginx-pod created
root@k8s-master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   1/1     Running   0          14s
root@k8s-master:~# kubectl expose pod nginx-pod \
--name=nginx-nodeport \
--port=80 \
--target-port=80 \
--type=NodePort
service/nginx-nodeport exposed
root@k8s-master:~# kubectl get svc nginx-nodeport
NAME             TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
nginx-nodeport   NodePort   10.105.75.52   <none>        80:30388/TCP   9s
root@k8s-master:~# kubectl run curl-pod \
--image=curlimages/curl \
--restart=Never \
-n test \
-- sleep 3600
pod/curl-pod created
root@k8s-master:~# kubectl get pods -n test
NAME       READY   STATUS    RESTARTS   AGE
curl-pod   1/1     Running   0          21s
```

- Verified successful communication in browser



```
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
~ $ exit
root@k8s-master:~#
```

- Created nginx Pod in the default namespace.
- Exposed nginx Pod using NodePort Service.
- Created curl client Pod in the test namespace.
- Accessed nginx using fully qualified Service DNS.
- Verified successful communication using curl.

## Task 5 – LimitRange Memory Ratio

To implement a Kubernetes LimitRange policy in the  namespace that restricts the maximum memory limit-to-request ratio to 2. To validate the policy by creating Pods with valid and mismatched memory requests and limits, ensuring Kubernetes rejects configurations that exceed the defined ratio.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-ratio
  namespace: performance-environment
spec:
  limits:
  - type: Container
    maxLimitRequestRatio:
      memory: "2"
~
~
~
~
~
```

```
root@k8s-master:~# kubectl create namespace performance-environment
namespace/performance-environment created
root@k8s-master:~# vi performance-limitrange.yaml
root@k8s-master:~# kubectl apply -f performance-limitrange.yaml
limitrange/memory-ratio created
root@k8s-master:~# kubectl describe limitrange memory-ratio -n performance-environment
Name:       memory-ratio
Namespace:  performance-environment
Type        Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio
----        --------  ---  ---  ---------------  -------------  -----------------------
Container   memory    -    -    -                -              2
root@k8s-master:~#
```

# -IMPORTANT CONCEPTS LEARNED TODAY

## 1. Namespace

- Namespace is used to logically separate resources inside a Kubernetes cluster.
- It helps organize resources for different environments.
- Examples: `dev-environment`, `prod-environment`, `staging-environment`, `test`.

## 2. ResourceQuota

- ResourceQuota controls the maximum resource consumption in a namespace.
- It can restrict the number of Pods, Services, PVCs, CPU and memory consumption.
- Example:
  - Maximum Pods: 3
  - Maximum Services: 2

## 3. Resource Requests and Limits

- Requests: Minimum resources Kubernetes uses when scheduling a Pod.
- Limits: Maximum resources a container is allowed to consume.
- CPU is measured in cores or millicores.
- Memory is measured in Mi or Gi.

## 4. LimitRange

- LimitRange controls resource requests and limits at namespace level.
- It can define default, minimum and maximum resource values.
- It helps prevent containers from requesting excessive resources.

## 5. Service DNS

- Kubernetes Services provide stable network access to Pods.
- DNS allows Pods to communicate using Service names.
- Pods can communicate across different namespaces.

## . IMPORTANT COMMANDS USED TODAY

### Namespace commands

| Command | Purpose |
|---|---|
| `kubectl create namespace NAME` | Create namespace |
| `kubectl get namespaces` | List namespaces |
| `kubectl get pods -A` | List Pods in all namespaces |
| `kubectl get pods -n NAME` | List Pods in a namespace |
| `kubectl delete namespace NAME` | Delete namespace |

### ResourceQuota commands

| Command | Purpose |
|---|---|
| `kubectl apply -f resource-quota.yaml` | Apply ResourceQuota |
| `kubectl get resourcequota -n NAME` | View quota |
| `kubectl describe resourcequota NAME -n NS` | Detailed quota information |

### LimitRange commands

| Command | Purpose |
|---|---|
| **`kubectl apply -f staging-limitrange.yaml`** | **Apply LimitRange** |
| **`kubectl get limitrange -n NAME`** | **List LimitRanges** |
| **`kubectl describe limitrange NAME -n NS`** | **Inspect LimitRange** |

### Pod resource commands

| Command | Purpose |
|---|---|
| `kubectl apply -f pod.yaml` | Create or update Pod |
| `kubectl get pods -n NAME` | Check Pod status |
| `kubectl describe pod POD -n NS` | View Pod details |
| `kubectl delete pod POD -n NS` | Delete Pod |

### Service and networking commands

| Command | Purpose |
|---|---|
| `kubectl expose pod nginx-pod --name=nginx-nodeport --port=80 --target-port=80 --type=NodePort` | Create NodePort Service |
| `kubectl get svc -A` | List all Services |
| `kubectl describe svc SERVICE` | Inspect Service |
| `kubectl exec -it POD -n NS -- sh` | Enter container |
| `kubectl exec -it POD -n NS -- curl URL` | Test connectivity from Pod |